

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
Facultad de Ciencias de la Computación y Diseño Digital
Carrera de Ingeniería en Software

Ingeniería de Requisitos (ISR-401)

Docente: Ing. Gleiston Guerrero Ulloa, PhD.
Período académico: 2026–2027 PPA
Paralelo: [4to Software A]

Herramientas CASE para Generación de Código: Visual Paradigm

Proyecto Fin de Curso: Sistema Inteligente de Mantenimiento de Palma Africana

Integrantes:

Castro Bajaña Ariel Omar
Mora Duarte Alex José
Villafuerte Rosero Allan Noé
Vera Gómez Anthony Alfredo

Resumen

Este trabajo presenta la aplicación del enfoque Model-Driven Development (MDD) al subsistema de mantenimiento fitosanitario del Sistema Inteligente de Mantenimiento de Palma Africana (SIMPA), utilizando Visual Paradigm 18.0 como herramienta CASE de generación de código. A partir de un modelo UML organizado en cuatro paquetes (SIMPA.Dominio, SIMPA.Aplicacion, SIMPA.Persistencia y SIMPA.Integraciones), se ejecutó una transformación modelo-a-texto (M2T) mediante el Instant Generator de la herramienta, produciendo código C# correspondiente a las ocho clases del dominio y a los elementos técnicos asociados (servicios, repositorios e interfaces).

El documento fundamenta teóricamente la decisión bajo los principios de MDE y MDA, justifica la selección de Visual Paradigm frente a alternativas como Enterprise Architect y GitHub Copilot, y examina los criterios éticos de transparencia, licenciamiento y confiabilidad involucrados en el uso de generadores de código. Como evidencia del cierre del ciclo de desarrollo, se documenta un caso de roundtrip: la incorporación del atributo fechaUltimoMantenimiento a la clase Palma, verificando su correcta propagación al código generado.

Los resultados confirman que la generación determinista de código a partir de modelos UML ofrece trazabilidad, reproducibilidad y verificabilidad superiores a los enfoques basados en IA generativa, constituyendo una base metodológica sólida para el desarrollo del subsistema evaluado.

Índice

1. Marco teórico	3
1.1. Model-Driven Engineering (MDE)	3
1.2. Model-Driven Architecture (MDA) y niveles de abstracción	3
1.3. Model-Driven Development (MDD)	3
1.4. Transformaciones M2M y M2T	3
1.5. Estado del arte y vigencia del enfoque	4
2. Criterios éticos en el uso de herramientas de generación de código	4
2.1. Transparencia: código generado vs. código escrito a mano	4
2.2. Licenciamiento del código generado	4
2.3. Sesgos y limitaciones de los generadores basados en IA generativa	4
3. Justificación de la selección de la herramienta	5
4. Descripción del subsistema del PFC modelado	6
5. Modelo UML de origen	6
6. Configuración del generador	8
6.1. Alcance del trabajo realizado	8
6.2. Configuración del generador	8
7. Resultado de la generación	8
7.1. Compilación y unidad demostrable	9
7.2. Correspondencia entre el modelo UML y el código	9
7.3. Evaluación de calidad del código generado	10
7.4. Relación con los requisitos de SIMPA	10
7.5. Política de regeneración	11
7.6. Evidencias de verificación	11
7.7. Conclusión de la verificación	11
8. Cierre del ciclo (roundtrip)	12
8.1. Cambio aplicado	12
9. Reparto del trabajo por integrante	13
10. Conclusiones	13
11. Referencias	13
11.1. Captura de evidencia 1	15
11.2. Captura de evidencia 2	15
11.3. Captura de evidencia 3	16
11.4. Captura de evidencia 4	16

1. Marco teórico

1.1. Model-Driven Engineering (MDE)

El Model-Driven Engineering (MDE) es el paradigma de ingeniería de software en el que los modelos dejan de ser documentación de apoyo y pasan a ser artefactos de primera clase del proceso de desarrollo: la fuente autoritativa a partir de la cual se derivan, mediante transformaciones automatizadas, las implementaciones del sistema [1]. Bajo este enfoque, el esfuerzo de ingeniería se desplaza de la escritura manual de código hacia la definición precisa de modelos y de las reglas que los transforman, lo que favorece la trazabilidad, la reutilización y la reducción de errores de transcripción entre el diseño y la implementación. Trabajos recientes confirman que este desplazamiento no es solo teórico: Bucaioni et al. [2], en una revisión sistemática multivocal sobre el uso del modelado en el desarrollo *low-code*, encuentran que las plataformas *low-code* actuales —pensadas para usuarios sin formación técnica profunda— se apoyan de forma consistente en principios de modelado propios del MDE, lo que evidencia la vigencia y expansión industrial del paradigma más allá de los entornos tradicionales de modelado UML.

1.2. Model-Driven Architecture (MDA) y niveles de abstracción

La Model-Driven Architecture (MDA) es la iniciativa concreta del Object Management Group (OMG) que operacionaliza el MDE mediante tres niveles de abstracción [3]:

- **CIM (Computation Independent Model):** modelo de dominio y de negocio, sin referencia a cómo el sistema será construido. En el caso del PFC, corresponde al proceso de negocio de monitoreo y mantenimiento de las palmeras.
- **PIM (Platform Independent Model):** modelo de la lógica y estructura del sistema, independiente de la plataforma tecnológica de despliegue. Es el nivel en el que se sitúa el diagrama de clases construido en Visual Paradigm (entidades como *Palma*, sus atributos y relaciones).
- **PSM (Platform Specific Model):** modelo derivado del PIM, ya ligado a una plataforma, lenguaje o framework concretos (el código Java/otro lenguaje objetivo generado por Visual Paradigm).

Entre estos niveles median transformaciones que preservan, idealmente, la semántica del modelo de origen a medida que se añade detalle específico de plataforma [3, 4].

1.3. Model-Driven Development (MDD)

El MDD es la aplicación práctica del MDE al ciclo de vida del software: los modelos guían la construcción del sistema y el código fuente se obtiene, total o parcialmente, mediante generadores automáticos en lugar de escribirse a mano [5]. El valor del MDD no está solo en la automatización, sino en que obliga a que el modelo sea preciso y ejecutable-por-transformación: es justamente esa propiedad la que este equipo explota al usar el *Instant Generator* de Visual Paradigm sobre el subsistema *Palma* del PFC, descrito en la Sección 4 de este documento.

1.4. Transformaciones M2M y M2T

Dentro del ciclo MDA/MDD existen dos familias de transformación:

- **Modelo-a-modelo (M2M):** produce un modelo destino (p. ej. un PSM) a partir de un modelo de origen (p. ej. un PIM), ambos conformes a sus respectivos metamodelos. El estándar de referencia del OMG es QVT (Query/View/Transformation) v1.3 [6].

- **Modelo-a-texto (M2T):** produce artefactos textuales —código fuente, configuración, documentación— a partir de un modelo. El estándar de referencia es MOFM2T v1.0 [7].

La generación de código realizada en este trabajo corresponde a una transformación M2T: el diagrama de clases UML no se transforma en otro modelo, sino directamente en el texto fuente del lenguaje objetivo, mediante la plantilla del *Instant Generator* de Visual Paradigm.

1.5. Estado del arte y vigencia del enfoque

El campo de las transformaciones de modelos no es incipiente ni marginal. Kahani et al. [8] catalogan más de 60 herramientas de transformación en uso, y Bucchiarone et al. [9] documentan retos abiertos vigentes en escalabilidad, interoperabilidad y adopción industrial del MDE. Más recientemente, Bucaioni, Cicchetti y Ciccozzi [2] realizan una revisión sistemática que confirma el crecimiento sostenido del interés académico e industrial por el modelado como motor de generación de software, situando explícitamente a herramientas CASE como Visual Paradigm dentro de esta tendencia consolidada y no como un caso aislado o discontinuado.

2. Criterios éticos en el uso de herramientas de generación de código

Junto con el rigor técnico del ciclo MDD, es necesario considerar los criterios éticos aplicados durante la generación de código con Visual Paradigm. Se abordan tres ejes.

2.1. Transparencia: código generado vs. código escrito a mano

Todo el código producido por el *Instant Generator* de Visual Paradigm reside exclusivamente en la carpeta **generado/** del repositorio del equipo, separada de cualquier ajuste manual posterior. Cuando fue necesario completar a mano una parte que el generador no cubre, dicho fragmento se documenta explícitamente en la tabla de correspondencia modelo → código de la Sección 9, indicando el elemento del modelo que lo origina y la razón por la que requirió intervención manual. Esta separación es una condición ética mínima para la validez de la evidencia presentada: presentar código escrito a mano como si fuera generado por la herramienta —o viceversa— compromete tanto la fiabilidad de la demostración en vivo como la trazabilidad exigida entre el modelo UML y el artefacto de software resultante.

2.2. Licenciamiento del código generado

Visual Paradigm es software propietario, distribuido bajo licencia comercial/educativa. Según los términos de uso habituales de este tipo de herramientas CASE, el código fuente producido por el *Instant Generator* no hereda las restricciones de la licencia de la herramienta que lo generó: el resultado de la generación pertenece a quien opera la herramienta sobre su propio modelo. No obstante, el equipo no verificó este punto contra el contrato de licencia específico de la versión educativa utilizada, por lo que se recomienda confirmarlo antes de cualquier publicación o reutilización del código fuera del ámbito estrictamente académico de este proyecto. El equipo declara que el repositorio del PFC (<https://github.com/AlanNVR/Grupo-C>) se publica con fines exclusivamente académicos, y que las plantillas de generación propias de Visual Paradigm no se redistribuyen fuera del alcance permitido por la licencia educativa de la Universidad.

2.3. Sesgos y limitaciones de los generadores basados en IA generativa

Conviene distinguir dos familias de "generación automática de código" que suelen confundirse:

1. **Generación determinista basada en transformación de modelos (MDA/MDD):** la usada en este trabajo con Visual Paradigm. El código se produce por reglas fijas (plantillas M2T) aplicadas de forma reproducible sobre un modelo explícito. Dado el mismo modelo y la misma plantilla, el resultado es siempre el mismo, y cada línea generada es trazable a un elemento concreto del diagrama de clases.
2. **Generación mediante IA generativa (LLMs, p. ej. GitHub Copilot):** el código se produce por inferencia probabilística sobre un corpus de entrenamiento, sin garantía de reproducibilidad ni de trazabilidad a una especificación formal.

La literatura reciente documenta riesgos concretos de la segunda familia que no aplican al enfoque MDA/Visual Paradigm usado en el PFC. Pearce et al. [10], en un estudio empírico sobre 1 689 programas producidos por GitHub Copilot, encontraron que aproximadamente el 40 % contenía vulnerabilidades de seguridad explotables, resultado directamente atribuible a que el modelo generativo aprende patrones —incluyendo patrones inseguros— de su corpus de entrenamiento, sin ninguna garantía formal de corrección. Ebert y Louridas [11] añaden que, más allá de la seguridad, el uso de IA generativa en la industria del software plantea interrogantes abiertos sobre productividad real, gobernanza del riesgo y procedencia del código sugerido, precisamente por la ausencia de un modelo fuente auditable del cual derive la sugerencia.

Frente a esto, la generación determinista de Visual Paradigm no sufre de alucinaciones ni de incertidumbre sobre el origen del código, porque no existe entrenamiento estadístico de por medio: cada clase, atributo y método generado proviene de un elemento explícito del diagrama UML, verificable y editable por el equipo. Esta propiedad es la que permite ejecutar de forma confiable el *roundtrip* controlado exigido en la Sección 8 de este documento, algo que un generador basado en IA generativa no podría garantizar de forma determinista. Por esta razón —y no solo por conveniencia técnica— el equipo considera que la elección de Visual Paradigm es también una decisión éticamente más segura para un artefacto académico que debe ser reproducible y verificable.

3. Justificación de la selección de la herramienta

La herramienta seleccionada fue Visual Paradigm 18.0 porque ofrece un flujo directo y reproducible desde clases UML hacia código C#, mediante su *Instant Generator*. La documentación oficial permite seleccionar el lenguaje, las clases, la ruta de salida y, cuando sea necesario, un directorio de plantillas personalizadas [12]. Esta capacidad satisface el propósito de la actividad: demostrar una transformación modelo-a-texto sobre un modelo perteneciente al PFC y no producir código aislado sin correspondencia con UML.

Visual Paradigm también permite crear perfiles propios, definir estereotipos y valores etiquetados, y aplicarlos a elementos concretos del modelo [13, 14]. En SIMPA esta capacidad se utiliza para separar entidades del dominio, servicios, repositorios, integraciones externas y objetos de transferencia, conservando una relación explícita entre cada elemento UML y el artefacto C# que debe generar. Los estereotipos se consideran metadatos de clasificación; solo modifican el código cuando la plantilla de generación consulta sus valores, por lo que no se atribuye al estereotipo **Entity** un comportamiento de persistencia automático que no haya sido configurado.

Enterprise Architect fue considerado como alternativa válida, ya que su documentación oficial también contempla generación, ingeniería inversa, sincronización y personalización de plantillas [15]. Sin embargo, migrar en esta etapa obligaría a reconstruir o convertir el proyecto nativo, repetir la configuración del perfil y rehacer evidencias ya producidas. El repositorio del equipo ya contiene el archivo `SIMPA_MDD_Grupo_C.vpp`, capturas progresivas del modelado y una validación sin advertencias. Por tanto, la decisión no se basa en que Enterprise Architect carezca de funciones MDD, sino en que Visual Paradigm reduce el riesgo de migración y mantiene la trazabilidad de los artefactos existentes.

GitHub Copilot tampoco se seleccionó como mecanismo principal porque su función es proponer sugerencias de código dentro del editor, que el desarrollador puede aceptar o descartar, y no transformar de forma determinista un modelo UML de origen [16]. Puede emplearse como apoyo secundario, pero no reemplaza el ciclo modelo–generación–verificación–regeneración exigido por la rúbrica.

En consecuencia, Visual Paradigm representa la alternativa de menor riesgo para el equipo: trabaja con el formato nativo ya versionado, genera C#, admite perfiles UML, permite integración con Visual Studio y ofrece mecanismos de actualización del modelo desde el código [17, 18].

4. Descripción del subsistema del PFC modelado

El modelo de origen corresponde al subsistema de mantenimiento y monitoreo fitosanitario del Sistema Inteligente de Mantenimiento de Palma Africana. Su propósito es registrar la estructura de plantaciones, lotes y palmas; identificar al trabajador responsable; almacenar las labores ejecutadas; asociar diagnósticos a una palma; y producir alertas cuando se detecta una condición relevante.

El alcance mantiene trazabilidad con los requisitos del ERS de SIMPA: gestión de plantaciones y lotes (RF-002), gestión de personal (RF-003), registro de labores agrícolas (RF-004), monitoreo fitosanitario (RF-005), análisis y diagnóstico (RF-007 y RF-008), generación de alertas (RF-012) e historial por lote (RF-020). No incluye control automático de riego, operación de drones, gestión contable ni comercialización de la producción, ya que esas capacidades están excluidas del PFC.

La clase **Palma** funciona como entidad central. Cada palma pertenece a un lote, puede recibir labores y puede acumular diagnósticos. Las responsabilidades de coordinación se ubican en **MantenimientoService**, mientras el acceso a datos y la integración con el servicio de análisis se representan mediante interfaces, evitando mezclar la lógica del dominio con detalles de infraestructura.

5. Modelo UML de origen

El modelo se organizó en cuatro paquetes: **SIMPA.Dominio**, **SIMPA.Aplicacion**, **SIMPA.Persistencia** y **SIMPA.Integraciones**. El núcleo contiene ocho clases del dominio: **Plantacion**, **Lote**, **Palma**, **Usuario**, **Trabajador**, **Labor**, **Diagnostico** y **Alerta**. Todas presentan atributos tipados, visibilidad explícita y operaciones con parámetros y tipo de retorno. Como elementos técnicos se incorporan **MantenimientoService**, **IPalmaRepository**, **IServicioAnalisisIA** y **ResultadoAnalisisDT0**.

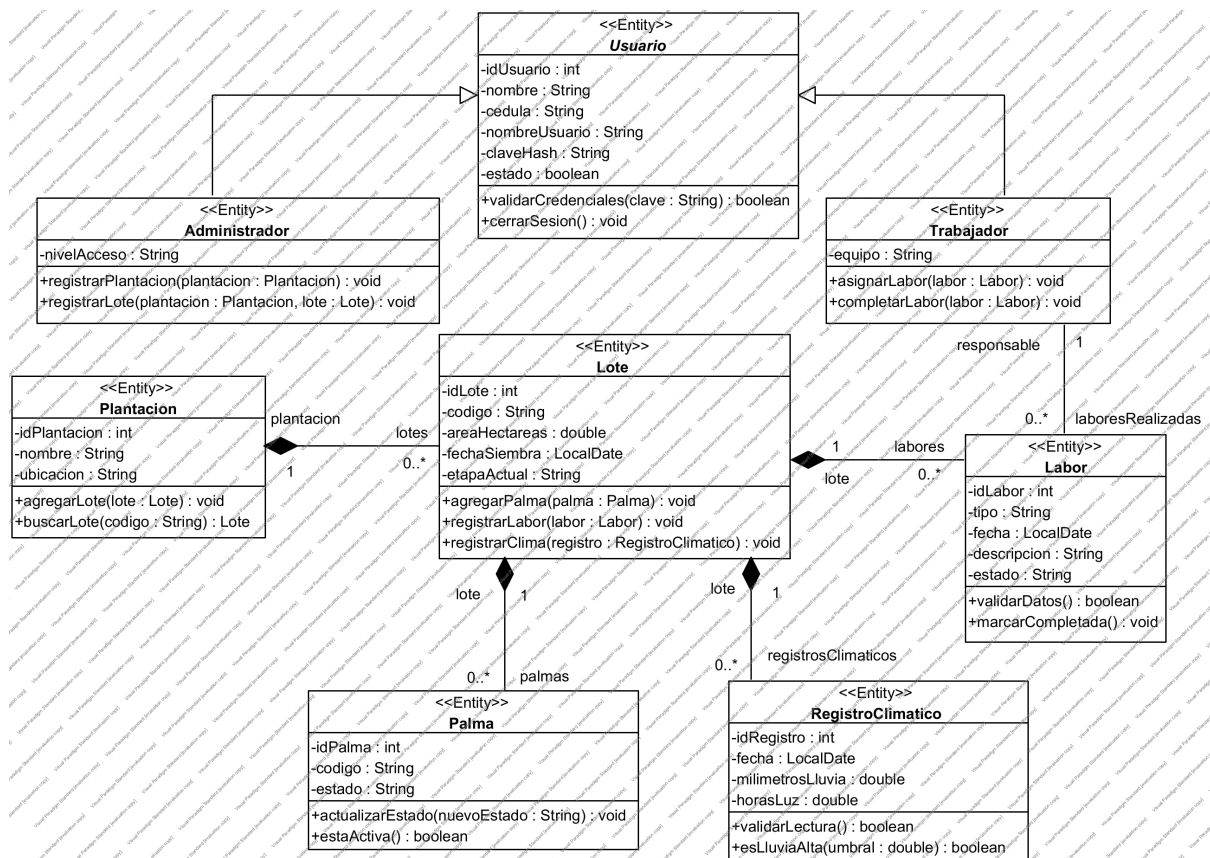


Figura 1: Diagrama de clases de origen del subsistema de mantenimiento de SIMPA.

Cuadro 1: Relaciones principales y multiplicidades del modelo de origen.

Origen	Relación	Destino	Multiplicidad
Plantacion	Composición	Lote	1 a 1..*
Lote	Composición	Palma	1 a 1..*
Lote	Asociación	Labor	1 a 0..*
Trabajador	Asociación	Labor	1 a 0..*
Palma	Asociación	Diagnostico	1 a 0..*
Diagnostico	Asociación	Alerta	1 a 0..1
Trabajador	Generalización	Usuario	Herencia
MantenimientoService	Dependencia	Repositorio/servicio	Uso
		IA	

Cuadro 2: Aplicación del perfil UML de SIMPA.

Estereotipo	Metaclass	Elementos
Entity	Class	Plantacion, Lote, Palma, Usuario, Trabajador, Labor, Diagnostico y Alerta
Service	Class	MantenimientoService
Repository	Interface	IPalmaRepository
ExternalService	Interface	IServicioAnalisisIA
DTO	Class	ResultadoAnalisisDTO

La composición expresa que un lote no existe fuera de su plantación y que una palma pertenece a un lote. Las asociaciones de labores y diagnósticos conservan historial sin forzar que cada

registro sea único. La multiplicidad 0..1 entre **Diagnostico** y **Alerta** permite que un diagnóstico normal no genere alerta, mientras uno crítico sí puede hacerlo.

El archivo nativo se conserva en `semana_12/02_Modelo/SIMPA_MDD_Grupo_C.vpp`. Para evitar alterar los diagramas previos, la vista refinada debe agregarse como `MDD_Origen_SIMPA`. El modelo se valida con Model Quality antes de ejecutar el generador; la comprobación automática de calidad identifica posibles defectos directamente sobre los elementos del diagrama [19].

Cuando se realice ingeniería inversa, el resultado se ubicará en un paquete separado `SIMPA.PSM.Implementacion`. Visual Paradigm permite actualizar modelos desde clases C# mediante su integración con Visual Studio [18]. De esta forma, los cambios del código pueden compararse con el modelo de origen sin sobrescribir de manera automática la línea base validada.

6. Configuración del generador

6.1. Alcance del trabajo realizado

Esta sección documenta la configuración del generador de código de Visual Paradigm, la generación de las clases Java derivadas del modelo UML de **SIMPA: Sistema Inteligente de Mantenimiento de Palma Africana**, la compilación del resultado y la ejecución de una unidad demostrable. El trabajo se realizó sobre el proyecto `SIMPA_MDD_Grupo_C.vpp` y no guarda relación con sistemas de gimnasio.

El modelo utilizado contiene ocho clases de dominio, todas clasificadas con el estereotipo **Entity**: **Usuario**, **Administrador**, **Trabajador**, **Plantacion**, **Lote**, **Palma**, **Labor** y **RegistroClimatico**.

6.2. Configuración del generador

El generador instantáneo de Visual Paradigm se configuró con Java como lenguaje objetivo y UTF-8 como codificación. Se activó la generación de getters y setters, operaciones de asociación, operaciones adicionales de colecciones, genéricos y archivo de construcción Ant. Las asociaciones múltiples se implementaron mediante `ArrayList`, mientras que los atributos de fecha se representaron mediante `java.time.LocalDate`. La compatibilidad se configuró para JDK 8 o superior.

Cuadro 3: Parámetros principales del generador de código

Parámetro	Configuración aplicada
Lenguaje y codificación	Java y UTF-8
Prefijos	Atributos: <code>_</code> ; parámetros: <code>a</code> ; variables locales: <code>l</code>
Asociaciones múltiples	<code>ArrayList</code> con genéricos
Versión de JDK	8.0 o superior
Archivo de construcción	Generación de <code>build.xml</code> activada
Confirmación de sobrescritura	Activada
Carpeta de salida	<code>05_Generado/codigo_generado</code>

Como política preventiva, se utilizó la vista previa antes de la generación y se mantuvo activa la confirmación antes de sobrescribir archivos. La generación finalizó correctamente al 100 %.

7. Resultado de la generación

Visual Paradigm produjo el archivo `build.xml` y ocho archivos Java dentro del paquete `ec.edu.uteq.simpa.modelo`. La estructura obtenida fue la siguiente:

```
codigo_generado/
|-- build.xml
```

```

'-- ec/edu/uteq/simpa/modelo/
    |-- Administrador.java
    |-- Labor.java
    |-- Lote.java
    |-- Palma.java
    |-- Plantacion.java
    |-- RegistroClimatico.java
    |-- Trabajador.java
'-- Usuario.java

```

La vista previa permitió comprobar que `LocalDate` se importó correctamente, que las multiplicidades de varios elementos se convirtieron en colecciones `ArrayList`, que se generaron getters y setters y que las referencias entre clases se conservaron en el código.

7.1. Compilación y unidad demostrable

La primera validación consistió en compilar las ocho clases exactamente como fueron generadas, sin modificaciones directas. La compilación concluyó sin errores y produjo ocho archivos `.class`.

Para cumplir con la ejecución de una unidad demostrable se creó código manual en una carpeta separada:

```

codigo_manual/
'-- ec/edu/uteq/simpa/demo/
    |-- Main.java
'-- LoteCrud.java

```

`Main.java` instancia y relaciona objetos del dominio de SIMPA. La demostración verifica la herencia de `Administrador` y `Trabajador`, el uso de getters y setters, las asociaciones entre plantación, lote, palma, labor y registro climático, y el manejo de fechas mediante `LocalDate`.

Además, `LoteCrud.java` implementa en memoria las operaciones Create, Read, Update y Delete para la entidad `Lote`. La compilación conjunta produjo diez archivos `.class`: ocho clases generadas, `Main.class` y `LoteCrud.class`. La ejecución confirmó el registro, consulta, actualización y eliminación de un lote, así como la verificación final de que el objeto dejó de existir.

Los comandos completos, las rutas de ejecución y los resultados esperados se encuentran documentados en el archivo `README.md` de la carpeta `05_Generado`; por esta razón no se repiten en esta sección del informe.

7.2. Correspondencia entre el modelo UML y el código

Cuadro 4: Correspondencia entre el modelo UML y el código generado, parte 1

Clase UML	Archivo generado	Correspondencia observada
Usuario	Usuario.java	Clase abstracta con atributos privados, getters y setters. Contiene dos operaciones sin implementación.
Administrador	Administrador.java	Hereda de <code>Usuario</code> mediante <code>extends</code> e incorpora el atributo <code>nivelAcceso</code> .
Trabajador	Trabajador.java	Hereda de <code>Usuario</code> , incorpora <code>equipo</code> y una colección de labores realizadas.
Plantacion	Plantacion.java	Conserva sus atributos y la composición múltiple con <code>Lote</code> mediante una colección.

Cuadro 5: Correspondencia entre el modelo UML y el código generado, parte 2

Clase UML	Archivo generado	Correspondencia observada
Lote	Lote.java	Incluye sus atributos, <code>LocalDate</code> y las relaciones con plantación, palmas, labores y registros climáticos.
Palma	Palma.java	Conserva sus atributos y la referencia simple hacia <code>Lote</code> .
Labor	Labor.java	Conserva atributos, fecha, estado y referencias hacia <code>Lote</code> y <code>Trabajador</code> .
RegistroClimatico	RegistroClimatico.java	Conserva fecha, lluvia, horas de luz y referencia hacia <code>Lote</code> .

La correspondencia es directa: cada clase UML produjo un archivo Java con el mismo nombre, mientras que la herencia, los tipos de fecha y las multiplicidades principales se trasladaron al código.

7.3. Evaluación de calidad del código generado

Tomando como referencia criterios de calidad de producto de ISO/IEC 25010, se obtuvieron los siguientes resultados:

- **Organización y trazabilidad:** el paquete y los nombres de archivo mantienen una correspondencia clara con el modelo UML, lo que facilita localizar el origen de cada elemento.
- **Encapsulación:** los atributos ordinarios son privados y cuentan con getters y setters; sin embargo, varias referencias y colecciones de asociaciones fueron generadas como campos públicos, lo que reduce el control sobre el estado interno.
- **Legibilidad:** los nombres representan correctamente conceptos del dominio de SIMPA y los comentarios describen el propósito de las operaciones.
- **Mantenibilidad:** la separación entre código generado y código manual disminuye el riesgo de perder cambios durante una regeneración. Los cambios estructurales deben realizarse primero en el modelo UML.
- **Herencia y asociaciones:** la jerarquía de usuarios y las relaciones principales se conservaron correctamente. No obstante, la consistencia bidireccional de las asociaciones debe mantenerse manualmente.
- **Compatibilidad:** el código se configuró para Java 8 o superior y fue compilado y ejecutado correctamente con Java 21.0.9 LTS.
- **Limitaciones:** no se generaron reglas de negocio completas, validaciones, persistencia, manejo de errores ni pruebas unitarias. El estereotipo `Entity` funciona como clasificación UML y no produjo anotaciones JPA.

En términos generales, el resultado constituye una base estructural válida y trazable, pero no una aplicación terminada. La generación automática reduce trabajo repetitivo, aunque requiere desarrollo manual para completar el comportamiento funcional.

7.4. Relación con los requisitos de SIMPA

Las clases generadas ofrecen soporte estructural para la gestión de usuarios, plantaciones, lotes, palmas, labores agrícolas y registros climáticos. Las asociaciones permiten representar la

relación entre un trabajador y sus labores, entre una plantación y sus lotes, y entre un lote y sus palmas, labores y datos climáticos.

Esta correspondencia es principalmente estructural. El cumplimiento completo de los requisitos funcionales dependerá de implementar las operaciones pendientes, las validaciones, la persistencia y las interfaces que se definan en iteraciones posteriores.

7.5. Política de regeneración

Para evitar pérdidas de información y mantener la trazabilidad se estableció la siguiente política:

1. Utilizar la vista previa antes de cada generación.
2. Mantener activada la confirmación antes de sobrescribir archivos.
3. Realizar en el modelo UML los cambios de clases, atributos, tipos, herencia y relaciones.
4. No editar directamente el código generado cuando el cambio pueda realizarse desde el modelo.
5. Mantener separado el código generado del código manual.
6. Compilar y ejecutar nuevamente la unidad demostrable después de cada regeneración.
7. Validar localmente antes de copiar archivos al repositorio.
8. Excluir carpetas de compilación y archivos `.class` del repositorio.
9. Conservar evidencias de configuración, generación, compilación y ejecución.
10. Registrar los cambios significativos mediante commits separados y trazables.

7.6. Evidencias de verificación

Las evidencias asociadas a esta sección se organizaron de la siguiente manera:

Cuadro 6: Evidencias de configuración, generación y verificación

Evidencias	Contenido demostrado
EV14-EV16	Vista previa de archivos, log de generación y revisión del código de Lote .
EV17-EV19	Generación completada, archivos Java obtenidos y estructura de salida con <code>build.xml</code> .
EV20	Compilación inicial de las ocho clases generadas.
EV21-EV23	Creación, compilación y ejecución de la unidad demostrable inicial.
EV24-EV26	Implementación, compilación y ejecución del CRUD manual de Lote .

7.7. Conclusión de la verificación

La configuración y generación de código se completaron correctamente. Las ocho clases producidas por Visual Paradigm compilaron sin modificaciones directas y conservaron los elementos principales del modelo UML. La unidad demostrable y el CRUD manual permitieron verificar el uso real de las entidades generadas sin mezclar el código manual con el código regenerable. Las limitaciones encontradas, especialmente las operaciones no implementadas y la exposición pública de asociaciones, deberán considerarse en futuras iteraciones de desarrollo.

8. Cierre del ciclo (roundtrip)

8.1. Cambio aplicado

Para evidenciar el cierre del ciclo se seleccionó la clase Palma, entidad central del dominio del sistema, y se le agregó el atributo fechaUltimoMantenimiento de tipo Date. Este cambio se eligió por ser no trivial (posee tipo definido y no corresponde a un simple renombrado) y por su pertinencia directa con el propósito del sistema: registrar la fecha de la última intervención de mantenimiento realizada sobre cada palma monitoreada. Tras aplicar el cambio en el modelo dentro de Visual Paradigm y validar su sintaxis, se regeneró el código utilizando la misma plantilla y parámetros ya configurados en la Sección 7, sin modificar la configuración del generador. El atributo nuevo se reflejó correctamente en la clase generada, junto con sus métodos de acceso correspondientes (getFechaUltimoMantenimiento / setFechaUltimoMantenimiento).

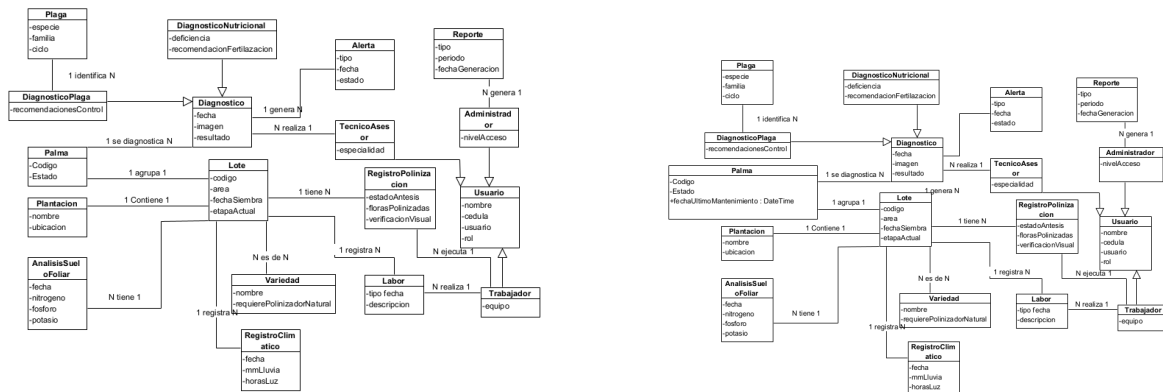


Figura 2: Modelo antes (izquierda) y después (derecha) del cambio.

```

using System;

public class Palma {
    private String codigo;
    private String estado;
    private Diagnostico_1_se_diagnostica_N;
    private Lote_1_agrupa_1;
}
    
```

```

using System;

public class Palma {
    private String codigo;
    private String estado;
    public DateTime FechaUltimoMantenimiento;
    private Diagnostico_1_se_diagnostica_N;
    private Lote_1_agrupa_1;
}
    
```

Figura 3: Código generado antes (izquierda) y después (derecha) del cambio.

Cuadro 7: Trazabilidad del cambio aplicado (roundtrip).

Elemento UML	Tipo de cambio	Artefacto afectado	Resultado
Palma.fechaUltimoMantenimiento	Alta de atributo (DateTime)	Palma.java	OK, campo + getter/setter generados

9. Reparto del trabajo por integrante

Cuadro 8: Distribución de responsabilidades del equipo.

Integrante	Responsabilidad principal
Ariel Castro Bajaan	Marco teórico y criterios éticos de IA
Alex Mora Duarte	Configuración del generador, proceso de generación y verificación
Allan Villafuerte Rosero	Modelado UML, justificación de la herramienta
Anthony Vera Gómez	Roundtrip, documento LaTeX, organización del repositorio

Enlace a commits por integrante: <https://github.com/AlanNVR/Grupo-C/commits/main/>

10. Conclusiones

El desarrollo de este trabajo permitió comprobar, de forma práctica y no solo teórica, la vigencia del paradigma Model-Driven Engineering como estrategia de generación de software. La aplicación del ciclo MDA sobre el subsistema de mantenimiento fitosanitario de SIMPA evidenció que un modelo UML preciso —con clases, atributos, relaciones y estereotipos correctamente definidos— puede constituirse en la fuente autoritativa a partir de la cual se deriva código fuente reproducible, trazable y libre de las incertidumbres propias de los generadores basados en IA generativa.

La selección de Visual Paradigm como herramienta CASE resultó adecuada para los fines del PFC: permitió ejecutar una transformación M2T determinista sobre el paquete SIMPA.Dominio, generar código C# correspondiente a las ocho clases del núcleo del dominio, y verificar la correspondencia exacta entre cada elemento del diagrama de clases y el artefacto generado. Asimismo, el cierre del ciclo roundtrip —mediante la incorporación del atributo fechaUltimoMantenimiento a la clase Palma— confirmó que el modelo y el código pueden mantenerse sincronizados sin pérdida de trazabilidad ni intervención manual sobre la lógica generada.

En el plano ético, el equipo constató que la transparencia entre código generado y código escrito a mano, así como la separación de responsabilidades sobre el licenciamiento de los artefactos producidos, son condiciones indispensables para sostener la validez académica de la evidencia presentada. La comparación con enfoques de generación basados en LLMs reforzó la conveniencia de un mecanismo determinista como el de Visual Paradigm para un ejercicio que exige reproducibilidad y verificación.

En conjunto, el proyecto demuestra que el subsistema modelado de SIMPA cumple con los requisitos funcionales de trazabilidad establecidos en el ERS (RF-002 a RF-020 relacionados), y que la metodología MDD aplicada constituye una base sólida para futuras iteraciones del sistema completo.

11. Referencias

Referencias

- [1] D. C. Schmidt, “Guest editor’s introduction: Model-driven engineering,” *Computer*, vol. 39, no. 2, pp. 25–31, 2006.
- [2] A. Bucaioni, A. Cicchetti, and F. Ciccozzi, “Modelling in low-code development: a multi-vocal systematic review,” *Software and Systems Modeling*, vol. 21, no. 5, pp. 1959–1981, 2022.

- [3] Object Management Group, “Model driven architecture (mda) guide, revision 2.0,” Tech. Rep. ormsc/14-06-01, OMG, 2014.
- [4] Object Management Group, “Omg unified modeling language (omg uml), version 2.5.1,” Tech. Rep. formal/17-12-05, OMG, 2017.
- [5] B. Selic, “The pragmatics of model-driven development,” *IEEE Software*, vol. 20, no. 5, pp. 19–25, 2003.
- [6] Object Management Group, “Meta object facility (mof) 2.0 query/view/transformation specification, version 1.3,” Tech. Rep. formal/16-06-03, OMG, 2016.
- [7] Object Management Group, “Mof model to text transformation language (mofm2t), version 1.0,” Tech. Rep. formal/2008-01-16, OMG, 2008.
- [8] N. Kahani, M. Bagherzadeh, J. R. Cordy, J. Dingel, and D. Varró, “Survey and classification of model transformation tools,” *Software and Systems Modeling*, vol. 18, no. 4, pp. 2361–2397, 2019.
- [9] A. Bucchiarone, J. Cabot, R. F. Paige, and A. Pierantonio, “Grand challenges in model-driven engineering: an analysis of the state of the research,” *Software and Systems Modeling*, vol. 19, no. 1, pp. 5–13, 2020.
- [10] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, “Asleep at the keyboard? assessing the security of github copilot’s code contributions,” in *Proc. 43rd IEEE Symposium on Security and Privacy (SP)*, pp. 754–768, 2022.
- [11] C. Ebert and P. Louridas, “Generative ai for software practitioners,” *IEEE Software*, vol. 40, no. 4, pp. 30–38, 2023.
- [12] Visual Paradigm, “How to generate c# from uml (instant generator guide)?.” https://www.visual-paradigm.com/support/documents/vpuserguide/276/330/7334_instantgener.html. Consultado el 19 de julio de 2026.
- [13] Visual Paradigm, “Drawing a profile diagram.” https://www.visual-paradigm.com/support/documents/vpuserguide/26/534/28731_drawingaprof.html. Consultado el 19 de julio de 2026.
- [14] Visual Paradigm, “Applying stereotype to model element.” https://www.visual-paradigm.com/support/documents/vpuserguide/1283/177/6563_applystereot.html. Consultado el 19 de julio de 2026.
- [15] Sparx Systems, “Feature overview: Code engineering with enterprise architect.” https://sparxsystems.com/enterprise_architect_user_guide/17.1/modeling_domains/code_engineering_with_ea.html. Consultado el 19 de julio de 2026.
- [16] GitHub, “Responsible use of github copilot inline suggestions.” <https://docs.github.com/en/copilot/responsible-use/inline-suggestions>. Consultado el 19 de julio de 2026.
- [17] Visual Paradigm, “How to generate source code from uml in visual studio?.” https://www.visual-paradigm.com/support/documents/vpuserguide/2381/2383/66566_codegenerati.html. Consultado el 19 de julio de 2026.
- [18] Visual Paradigm, “How to reverse engineer uml model in visual studio?.” https://www.visual-paradigm.com/support/documents/vpuserguide/2381/2383/66573_reverseengin.html. Consultado el 19 de julio de 2026.

- [19] Visual Paradigm, “Model quality options.” https://www.visual-paradigm.com/support/documents/vpuserguide/2270/2278/59860_modelquality.html. Consultado el 19 de julio de 2026.

Anexos

11.1. Captura de evidencia 1

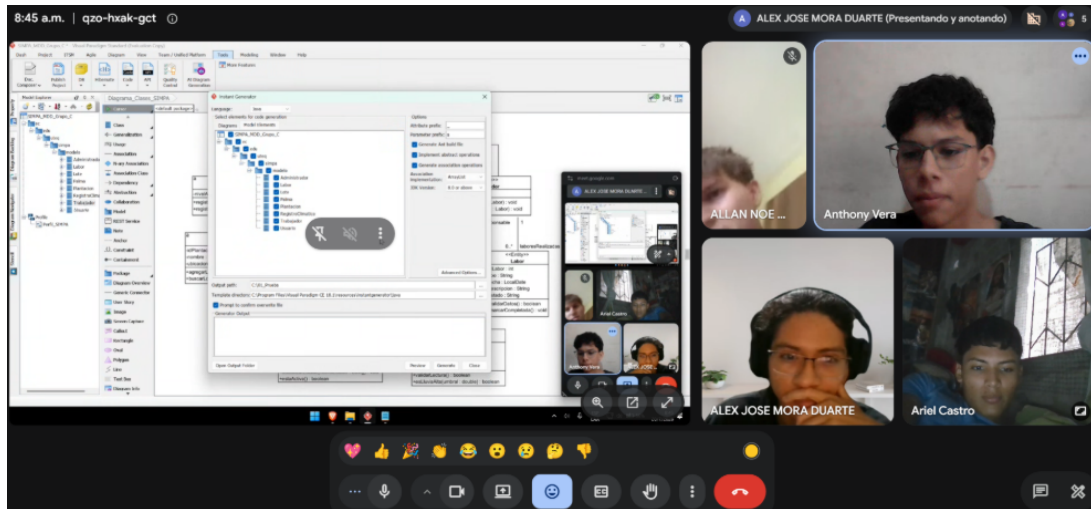


Figura 4: Captura de evidencia 1

11.2. Captura de evidencia 2

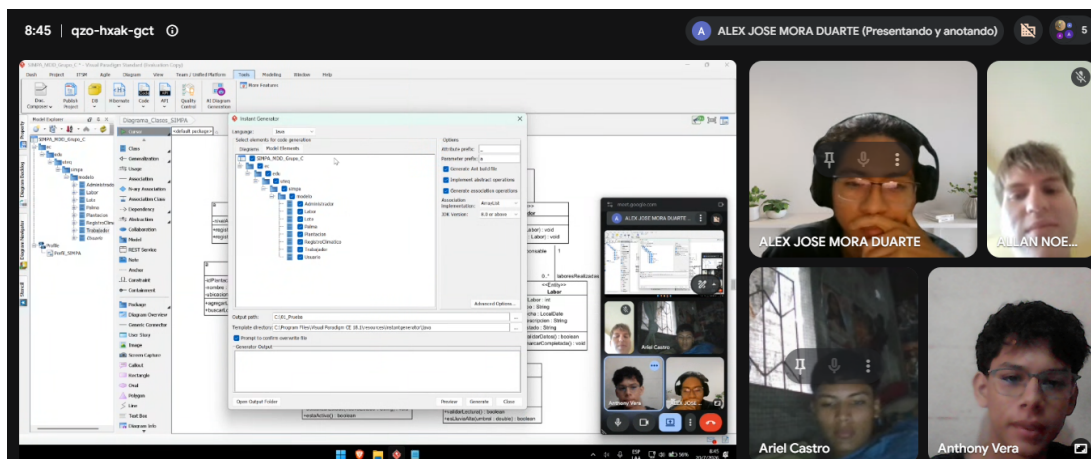


Figura 5: Captura de evidencia 2

11.3. Captura de evidencia 3

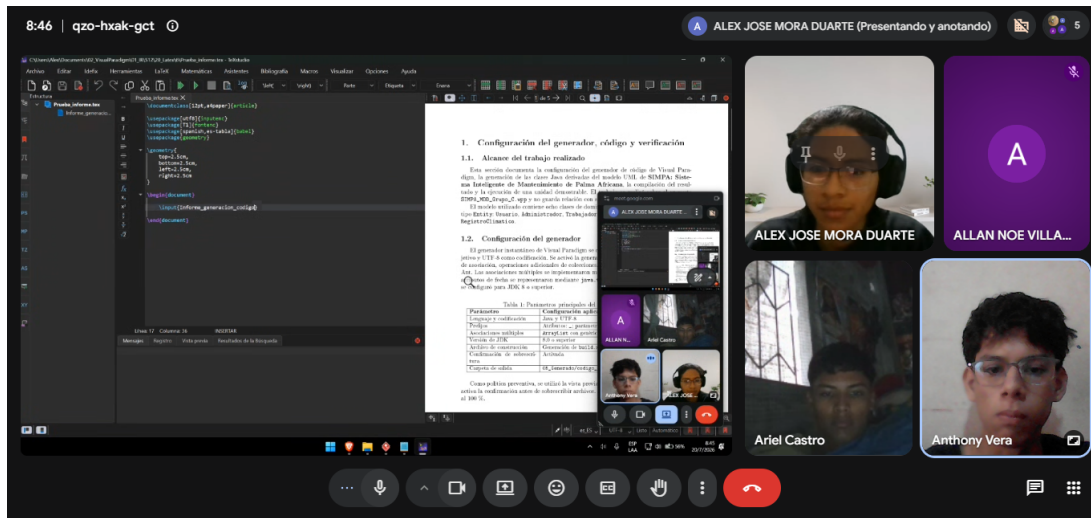


Figura 6: Captura de evidencia 3

11.4. Captura de evidencia 4

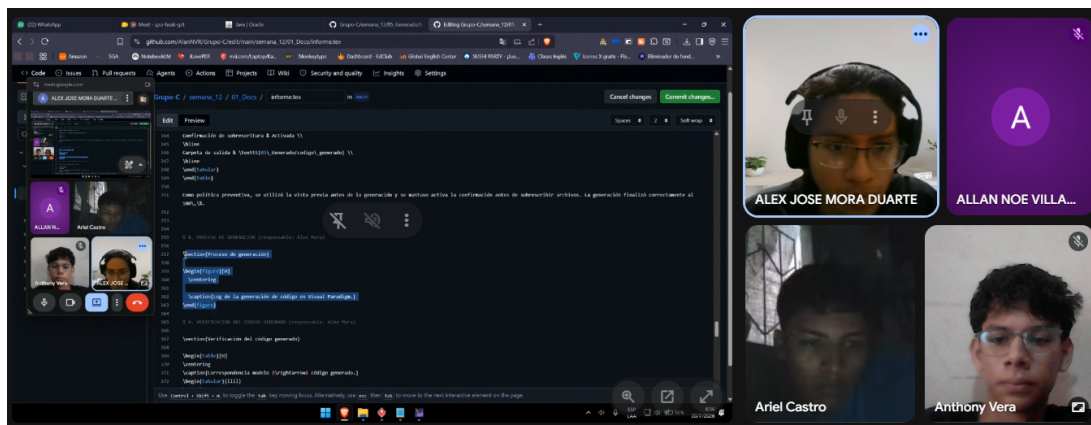


Figura 7: Captura de evidencia 4